**TU Dortmund**

# Fachprojekt Detektion von Sicherheitslücken

**M.Sc. Johannes Düsing, M.Sc. Anemone Kampkötter**

**JProf. Dr.-Ing. Ben Hermann**

## Aufgabe 2: Aufruf kritischer Methoden

## Ausgabe: 23.10.2024

## Besprechung: 06.11.2024

Die Java-Standardbibliothek stellt verschiedene Funktionen zur Verfügung, mit denen sicherheitsrelevante Mechanismen beeinflusst werden können. Unter anderem kann mit den *statischen* Methoden `System.getSecurityManager` und `System.setSecurityManager`[1] der aktuelle `SecurityManager` abgefragt bzw. neu gesetzt werden. Außerhalb von sicherheitsrelevanten Kontexten ist es selten nötig, diese Funktionen aufzurufen. Ihre Benutzung *kann* also auf sicherheitskritisches Verhalten hindeuten.

## 2.1 Analyseaufgabe

Implementieren Sie eine statische Programmanalyse, die herausfindet ob die oben genannten kritischen Methoden jemals bei einer Ausführung des analysierten Softwaresystems aufgerufen werden können.

Überlegen Sie sich zunächst auf konzeptueller Ebene, wie diese Problemstellung **sound** (d.h. ohne dass mögliche Aufrufe der Methoden nicht gefunden werden - keine *False Negatives*) gelöst werden kann. Überlegen Sie sich auch, ob Ihre Lösung *False Positives* erzeugen kann - also vor erreichbaren Aufrufen warnt, die in Wahrheit gar nicht vorkommen können.

Setzen Sie die Analyse dann konkret in Scala mithilfe von OPAL um.

## Technische Rahmenbedingungen:

- Die beiden eingangs genannten Methoden sind nicht die einzigen potentiell sicherheitsrelevanten Methodenaufrufe in Java. Ihre Analyse sollte für Nutzende konfigurierbar sein, sodass beliebige weitere Methodenaufrufe detektierbar sind.
- Stellen Sie sicher, dass Ihre Implementierung die geforderte Funktionalität korrekt umsetzt.
- Verwalten Sie die Code für Ihre Lösung in einem Git-Repository, dass über GitHub bzw. GitLab gehostet ist. Ermöglichen Sie uns bis spätestens 06.11.2024, 14:00 Uhr (lesenden) Zugriff. Strukturieren Sie das Repository so, dass es auch für alle zukünftigen Abgaben genutzt werden kann.

## Weitere Anforderungen:

- In seltenen Fällen kann es sinnvoll sein, Aufrufe der kritischen Methoden zu erlauben bzw. die Warnungen zu unterdrücken. Wie würden Sie diese Anforderung in Ihrer Implementierung umsetzen?

[1] `https://docs.oracle.com/javase/8/docs/api/java/lang/System.html#setSecurityManager-java.lang.SecurityManager-`